AI-Based Fleet Coordination – Practical Implementation Task

Dr Maria Chli, Dr Farzaneh Farhadi

Deadline: Monday 15 May, 5pm UK time

Objective

This task assesses your ability to design and implement basic **decentralised task allocation strategies** for autonomous baggage vehicles. The implementation requirements include:

1. Implementation of **auction** and/or **Q-Learning** fleet coordination algorithms in **Python**
2. Use of provided baseline code for environment setup and greedy allocation algorithm
3. Performance evaluation based on **task completion time** and **energy use** minimisation
4. Critical analysis of the strengths and limitations of your approaches

Problem Description

You are tasked with designing and implementing algorithms for assigning a fleet of autonomous vehicles to tasks in a dynamic environment. The goal is to allocate vehicles efficiently to tasks while minimising total time and energy consumption.

- Each vehicle has a position, a speed, and a battery level.
- Tasks arrive dynamically with a location, urgency level, and duration.
- Your implementation must include one or both of:
  - **Auction-based allocation**: Assign tasks to vehicles using an auction mechanism.
  - **Q-Learning allocation**: Implement a reinforcement learning approach to optimise task allocation over time.
- You will compare your solutions against a provided **greedy allocation** strategy.

Provided Code

You are given:

1. **Data generation**: Code to create random vehicle and task data.
2. **Greedy allocation**: A baseline method that assigns tasks to the closest available vehicle.
3. **Skeleton comparison code**: Functions to integrate and compare your allocation algorithm implementations.

Tasks

1. Implement an **Auction-based Allocation Strategy** (25 marks)
    - Implement vehicle bidding using an appropriate bidding function
    - Assign tasks to highest
    - Your bidding function should consider at least:
        - distance or travel time,
        - battery feasibility,
        - task urgency
    - Briefly justify:
        - why your bidding function was designed this way,
        - what information each vehicle is allowed to access,
        - and whether your method is fully decentralised or centrally coordinated with decentralised bidding
2. Implement a **Q-Learning Allocation Strategy** (35 marks)
    - Consider appropriate state space discretisation
    - Q-table implementation and training to optimise vehicle-task assignment decisions.
    - Clearly define and justify:
        - state representation,
        - action space,
        - reward function,
        - and exploration strategy
    - Explain what behavioural patterns the learned policy appears to learn
    - Briefly discuss one limitation of tabular Q-learning for this problem
3. **Performance Comparison** (20 marks)
    - Run experiments comparing Greedy/Auction/Q-Learning allocation
    - Metrics: Total time, energy consumption, task completion
    - Document these in a short (2-5 page) report covering:
        - Implementation approaches
        - Challenges faced
        - Performance reporting and observations
4. **Critical Reflection** (10 marks)
    - Discuss one scenario in your report where:
        - greedy allocation performs poorly,
        - your auction method performs poorly,
        - and your Q-learning method performs poorly
    - Explain:
        - what assumptions your methods make, and
        - what information is globally shared vs locally available
5. **Ensure Code Quality** (10 marks)
    - Readable, appropriate structure and documentation
    - Comments and explanations
    - Clear instructions for running experiments
6. **Extension Task** (20 bonus marks)

- Optional improvements:
  - Dynamic battery recharge
  - Vehicle prioritisation
  - Delay handling

7. **Submit Task**
   - Email your Python scripts and short report (2-5 pages) as a zip attachment, following the naming convention SurnameFirstName.zip, to f.farhadi@aston.ac.uk

Tools & Libraries

Recommended stack:

- Python 3.x
- Pandas (for handling dataset)
- NumPy (for calculations)
- Matplotlib (for basic visualisation)

Expected Time to Complete

You are expected to spend at most 8 hours on this task:

- 1 hour – Understanding given code and planning your approach
- 4 hours – Algorithm implementation
- 3 hours – Testing, analysis and reporting